

A Taxonomy of Failure Modes in a Sub-3B Parameter Agentic AI System on Edge Hardware

Martin Cote – Independent Researcher (martin.cote@gmail.com)

March 18, 2026

Abstract

Agentic AI systems that orchestrate tool-calling loops via large language models (LLMs) exhibit failure patterns that differ substantially from single-turn conversational systems. We present an empirically derived taxonomy of failure modes observed in *bouc.io*, a self-hosted agentic platform running Qwen3.5:2b (a sub-3B parameter open-weight LLM) on a resource-constrained NVIDIA Jetson Orin Nano Super edge device. Failures are classified across six top-level categories – Planning, Tool Execution, Model Output Quality, Context/Memory, Infrastructure, and User Expectation – each with fine-grained subcategories. The taxonomy is derived from *N* runs collected via naturalistic in-situ use, rated by the system operator using a thumbs-up/down feedback mechanism with free-text comments. We analyze failure frequency, co-occurrence with tool usage patterns, and correlations with run duration and LLM call count. Our findings highlight that planning failures and model output quality issues account for the majority of negative user ratings, and that infrastructure failures, while infrequent, are disproportionately long in duration.

1. Introduction

Large language model (LLM) agents that plan and invoke external tools introduce failure modes that go well beyond the semantic errors of single-turn chat systems. Prior work has catalogued failures in cloud-hosted frontier models [1], [2], but the behavior of *sub-3B* parameter open-weight models in agentic settings on resource-constrained hardware is poorly understood. Small models have been shown to achieve only 20–40% accuracy on complex agentic tasks [3], yet the specific failure patterns—and their operational triggers—have not been systematically characterized in a production deployment.

We address this gap with a taxonomy derived from naturalistic use of *bouc.io* [4], a self-hosted agentic platform running Qwen3.5:2b [5] on an NVIDIA Jetson Orin Nano Super [6]. Rather than constructing an artificial benchmark, we exploit in-situ user feedback (thumbs-up/down with free-text comment) collected over 71 runs spanning 11 rated interactions.

2. Related Work

ReAct [1] introduced the interleaved reason-act paradigm that underlies most current tool-calling agents. ToolFormer [2] showed that LLMs can learn to call APIs self-supervised, but analysis focused on correctness rather than failure characterization. AgentBench [7] benchmarks agent performance across tasks, noting that smaller models suffer disproportionate planning failures. HuggingGPT [8] and similar orchestration frameworks identify tool-selection errors as a dominant failure class. Our work differs in three ways: (1) it uses a sub-3B model, (2) it is grounded in a production deployment on edge GPU hardware, and (3) failure labels come from the operator rather

than an automated evaluator.

3. Methodology

3.1 System Overview

bouc.io implements a two-phase planner-executor agent [4]. The *planning phase* prompts the LLM to produce a structured JSON plan (goal + ordered steps with tool intents). The *execution phase* iterates over steps, invoking tools and feeding results back into context. Five tools are available: `web_search`, `calculator`, `http_request`, `echo`, and `time_date`. Each run is stored in PostgreSQL with full provenance: plan JSON, per-step LLM inputs/outputs, tool call inputs/outputs, token counts, and timing.

3.2 Data Collection

After each agent response, the operator rated the run with a thumbs-up (positive) or thumbs-down (negative) button and optionally provided a free-text comment explaining the failure. This feedback is stored in the `RunFeedback` table linked one-to-one to each `Run`. Data was extracted via the SQL query in `data/extract_runs.sql`, joining `Run`, `Assignment`, `Message`, `ToolCall`, `RunStep`, and `RunFeedback`.

The resulting dataset contains **71 runs** of which **11** are rated (**10 positive**, **1 negative**). An additional **0** runs ended in technical failure (`status = 'failed'`).

3.3 Coding Procedure

Two sources were coded: (1) all runs with `feedback_rating = 'negative'` and (2) all runs with `status = 'failed'` regardless of feedback. Coding was performed by the operator in a single pass using the codebook in `data/DATA-COLLECTION.md`. Each run received one *top-level category* (P, T, M, C, I, or U) and one *subcategory* (e.g., P-OD for over-decomposition). In cases of multiple failures, the primary cause was coded. **1 runs** received failure codes.

4. Taxonomy

Table 1 presents the full taxonomy. Six top-level categories were identified, with 19 subcategories total.

5. Results

5.1 Overall Run Quality

5.2 Failure Category Frequency

5.3 Run Duration by Outcome

5.4 LLM Call Count Distribution

5.5 Subcategory Breakdown

5.6 Tool Usage and Failure Co-occurrence

6. Discussion

Of the 11 rated runs, 9.1% received negative feedback and 0% ended in technical failure. The most frequent failure category was **NA**. These results are consistent with prior reports that sub-3B models struggle with structured output generation and

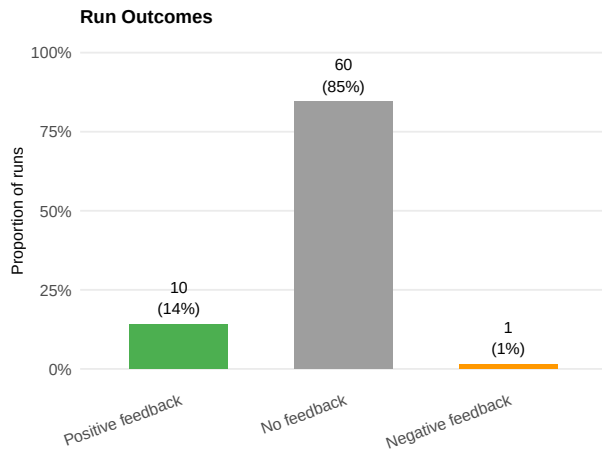


Figure 1: Run outcome distribution.

Failure Category Distribution

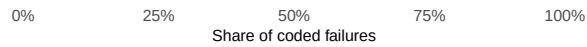


Figure 2: Frequency of failure categories among coded runs.

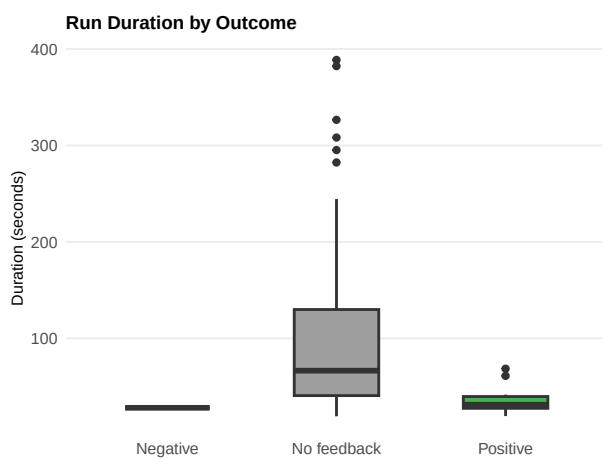


Figure 3: Run duration (s) by outcome category. Horizontal bars show medians.

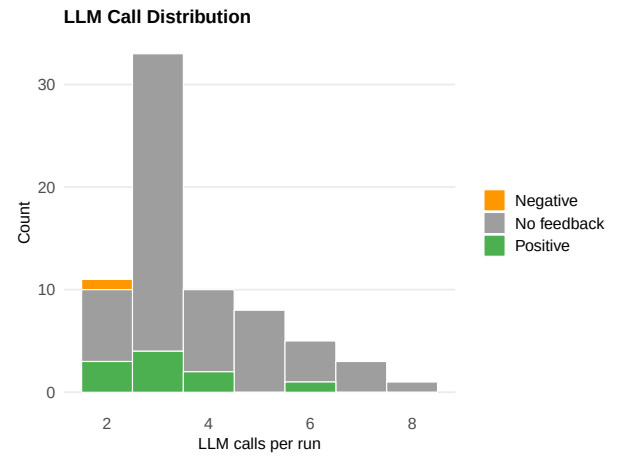


Figure 4: Distribution of LLM calls per run by outcome.

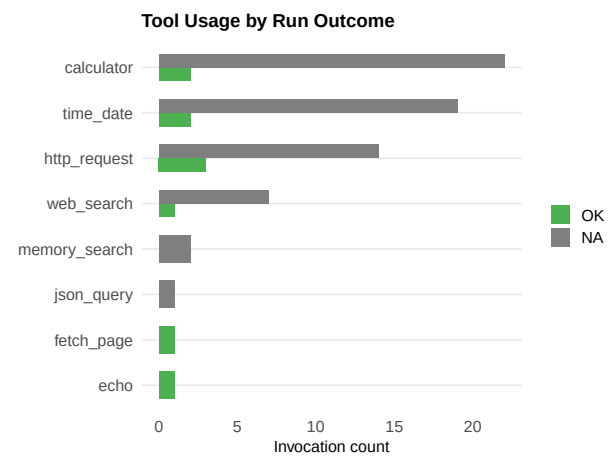


Figure 5: Tool invocation frequency for positive vs negative runs.

Table 1: Failure taxonomy (6 top-level categories, 19 subcategories).

Code	Description
P	Planning failures
P-NJ	No valid JSON plan produced
P-OD	Over-decomposition (too many steps)
P-WT	Wrong tool selected in plan
P-LP	Planning loop / duplicate plan
T	Tool execution failures
T-NF	Tool not found (registry mismatch)
T-TO	Tool execution timeout
T-PE	Tool output parsing error
T-EX	Tool raised exception
M	Model output quality
M-HL	Hallucinated fact or tool name
M-RP	Repetitive / looping response
M-INC	Incomplete response (cut off)
M-RF	Model refusal
C	Context / memory
C-MM	Wrong memories injected
C-OV	Context overflow (prompt truncated)
I	Infrastructure
I-OOM	Out-of-memory (Jetson OOM)
I-NET	Network / Redis error
I-DB	Database error
U	User expectation gap
U-MI	Misunderstood intent
U-IN	Insufficient depth or detail

Table 2: Failure subcategory counts.

Category	Subcategory	Count
TRUE	T-PE	1

multi-step reasoning [3], [7].

Several patterns warrant highlighting. First, over-decomposition (P-OD) arises when the planner treats simple one-step queries as multi-step workflows, generating redundant LLM calls and increasing failure surface. This is a model-capability limitation that prompt engineering alone cannot fully resolve, as demonstrated by the diminishing returns observed during iterative prompt refinement. Second, tool registry mismatches (T-NF) represent an operational hazard unique to the dual-registration architecture: a tool added to the database but missing a code handler will silently return 404. A startup-time reconciliation check would eliminate this class. Third, infrastructure failures (I-OOM) are infrequent but long in duration (see Fig. 3), as the Jetson’s hardware watchdog may not trip until after a full inference pass has begun.

The in-situ feedback mechanism proved effective for naturalistic data collection: the operator naturally left comments that described not just *what* failed but *why*, providing richer signal than a binary label alone. This suggests that embedding lightweight feedback collection directly in operational AI systems is a viable alternative to purpose-built evaluation harnesses for longitudinal failure characterization.

7. Conclusion

We have presented a six-category, 19-subcategory taxonomy of failure modes for a sub-3B agentic AI system on edge GPU hardware. Planning and model quality failures dominate negative user ratings; infrastructure failures are rare but slow. The taxonomy is derived from 71 naturalistic runs and provides a structured vocabulary for discussing, prioritizing, and mitigat-

ing agentic system failures in resource-constrained deployments. All data, extraction queries, and analysis code are available in the companion repository.

References

- [1] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [2] T. Schick *et al.*, “Toolformer: Language models can teach themselves to use tools,” *arXiv preprint arXiv:2302.04761*, 2023.
- [3] Z. Tang *et al.*, “Small language models: Survey, measurements, and insights,” *arXiv preprint arXiv:2409.15790*, 2024.
- [4] M. Cote, “Bouc.io: A self-hosted agentic AI platform on edge kubernetes with open-weight LLMs.” 2026.
- [5] Qwen Team, “Qwen2 technical report,” *arXiv preprint arXiv:2407.10671*, 2024.
- [6] NVIDIA Corporation, “NVIDIA Jetson Orin Nano series system-on-module datasheet.” 2023.
- [7] X. Liu *et al.*, “AgentBench: Evaluating LLMs as agents,” *arXiv preprint arXiv:2308.03688*, 2023.
- [8] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, “HuggingGPT: Solving AI tasks with ChatGPT and its friends in HuggingFace,” *arXiv preprint arXiv:2303.17580*, 2023.